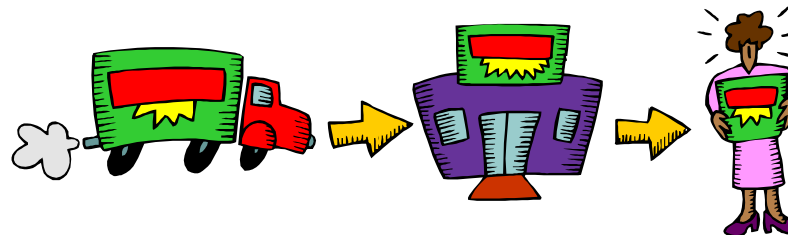


Công nghệ phần mềm

Kỹ nghệ Phần mềm (SE)



Nội dung

- Định nghĩa SE
- Các yếu tố cơ bản của SE
- Các giai đoạn phát triển của SE
- Mô hình phát triển chung
- Các thách thức cho phát triển PM
- Chi phí cho phát triển PM
- Software myths
- Một số vấn đề về đạo đức nghề nghiệp

ĐỊNH NGHĨA SOFTWARE ENGINEERING

SE = Software + Engineering

- Thế nào là software?
 - Mã nguồn + mã máy + cấu trúc dữ liệu + tài liệu
- Thế nào là Engineering (kỹ nghệ)?
 - Theo Oxford: “the activity of applying scientific knowledge to the design, building and control of machines, roads, bridges, electrical equipment, etc.”
- Thế nào là SE?

Một số định nghĩa về SE

- Nhiều nhà khoa học đưa ra các định nghĩa khác nhau
- Năm 1969, Bauer đưa ra một định nghĩa được coi là baseline:

“SE là thiết lập và sử dụng các nguyên lý công nghệ đúng đắn để được phần mềm một cách kinh tế, vừa tin cậy vừa làm việc hiệu quả trên các máy thực.”

Một số định nghĩa về SE

- SE còn là ngành công nghiệp liên quan đến mọi khía cạnh của việc phát triển phần mềm
 - thu thập, phân tích và đặc tả; thiết kế, lập trình, kiểm thử, chuyển giao, vận hành và bảo trì, ...
- SE đề xuất các chiến lược, các lý thuyết, phương pháp và công cụ nhằm phát triển các SP PM một cách hiệu quả

Tổng quan về SE

- Mục tiêu của SE
 - Làm thế nào để có được SP PM (trong môi trường đầy biến động, đầy thay đổi)
 - Có chất lượng cao (đủ các tính năng và dễ dùng)
 - Có thời gian phát triển ngắn
 - Có chi phí phát triển “thấp”
- Là một ngành của các nền kinh tế
 - Công nghiệp: phát triển, chuyển giao PM
 - Các trường ĐH, Viện NC: nghiên cứu, đề xuất các giải pháp cho các vấn đề mà công nghiệp đặt ra
 - R&D: mảng việc chung của hai đối tượng trên

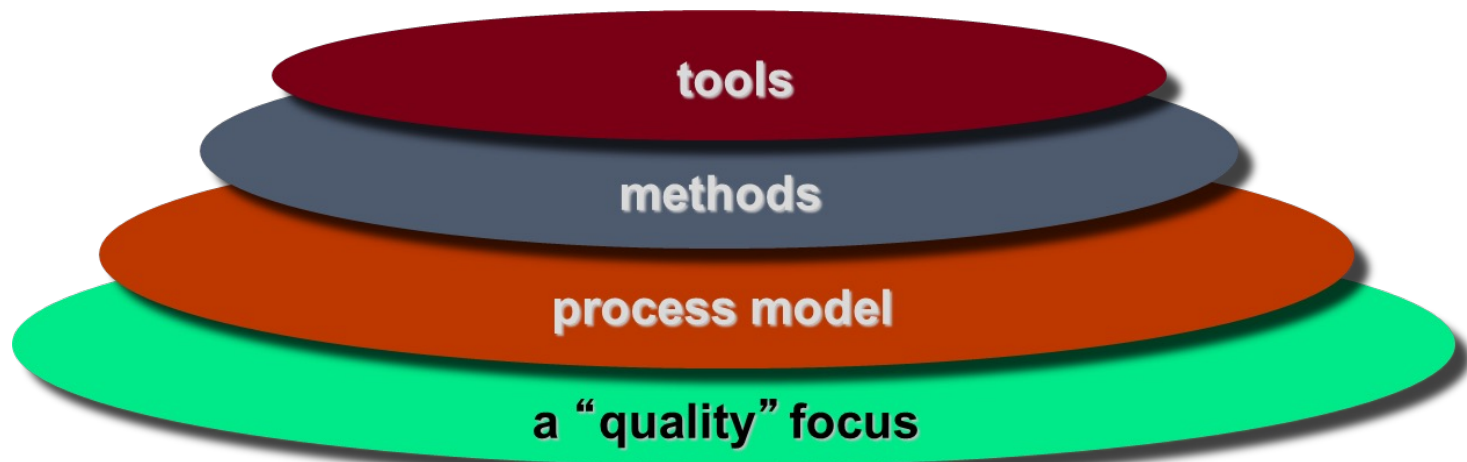
Nhà khoa học và Kỹ sư PM

- Nhà Khoa học
 - Đề xuất các giải pháp, các phương pháp và công cụ, ...
 - Có thời gian “vô hạn”
- Kỹ sư
 - Phát triển một giải pháp cho một vấn đề cụ thể với một khách hàng cụ thể
 - Sử dụng ngôn ngữ, công cụ, kỹ thuật và pp cụ thể
 - Làm việc trong nhiều miền ứng dụng khác nhau
 - Có thời hạn làm việc nghiêm ngặt, cụ thể
 - Luôn đối mặt với môi trường đầy biến động (thay đổi)

CÁC YẾU TỐ CƠ BẢN CỦA SE

Các lớp cơ bản

- SE được hợp thành từ các lớp (layer)
 - Lớp quality (lấy chất lượng làm nền tảng)
 - Lớp process (quy trình phát triển PM)
 - Lớp method (phương pháp – là làm mịn của lớp quy trình)
 - Lớp công cụ (để hỗ trợ thực hiện hai lớp process và lớp method)



Lớp process

- Liên quan đến quy trình quản lý và phát triển
 - Trình tự thực hiện các công việc (hay quy trình phát triển PM)
 - Phân tích, thiết kế, lập trình, kiểm thử, quản lý dự án, v.v.
 - Tài liệu, sản phẩm cần bàn giao và cách thức thực hiện
 - Mốc thời gian (milestones) và sản phẩm đưa ra (theo chuẩn)

Lớp method

- Lớp Process chỉ đề cập đến ý tưởng chung để xây dựng PM
- Lớp Method là làm mịn các bước trong ý tưởng của lớp Process
- Cụ thể
 - Làm mịn *Phân tích*: lớp method làm rõ về bước thu thập, phân tích, và đặc tả yêu cầu
 - Làm mịn *Thiết kế*: lớp method làm rõ về thiết kế hệ thống, thiết kế mức cao, thiết kế mức thấp, v.v.
 - Làm mịn *Lập trình*: lớp method làm rõ về phương pháp lập trình theo hướng cấu trúc, hay theo hướng đối tượng, v.v.
 - Làm mịn *Kiểm thử*: lớp method làm rõ kiểm thử theo hướng hộp đen, hộp trắng, hồi quy, v.v. như thế nào
 - Làm mịn *Quản lý dự án*
 - v.v.

Lớp tool

- Trợ giúp tự động/bán tự động các lớp Method và lớp Process
- Computer Aided Software Engineering – CASE, các công cụ trợ giúp các công đoạn khác nhau trong tiến trình phát triển phần mềm
 - IDE: Eclipse, Netbean, PyCharm, v.v.
 - UML design: Astah, v.v.
 - Testing: Junit, GoogleTest, v.v.
 - Quản lý dự án: MS Project, Faceworks, v.v.
 - Các công cụ đánh giá hiệu năng, khả năng chịu tải của HT: WebLoad, Apache Jmeter, v.v.

CÁC GIAI ĐOẠN PHÁT TRIỂN

CÁC GIAI ĐOẠN PHÁT TRIỂN

- ~70s: các phương pháp lập trình và cấu trúc dữ liệu
 - Tính mô đun
 - Sơ đồ khối, lập trình top down
 - Lập trình có cấu trúc
 - Phương pháp chia mô đun chương trình
 - Trừu tượng hóa dữ liệu

CÁC GIAI ĐOẠN PHÁT TRIỂN

- ~80s (nửa đầu)
 - Các phương pháp phát triển phần mềm
 - Công nghệ cơ sở dữ liệu (mô hình quan hệ)
 - Phân tích thiết kế hướng cấu trúc (biểu đồ luồng,...)
 - Các bộ công cụ phát triển (CASE tools)
 - Công cụ hỗ trợ phân tích, thiết kế
 - Công cụ soạn thảo, kiểm thử chương trình
 - Bắt đầu quan tâm đến quản lý
 - Các độ đo phần mềm

CÁC GIAI ĐOẠN PHÁT TRIỂN

- ~80s (nửa sau)
 - Hoàn thiện công nghệ cấu trúc
 - Chuẩn hóa các mô hình hướng cấu trúc
 - Case tools hoàn thiện, đạt mức tự động hóa cao
 - Các ngôn ngữ lập trình cao phát triển
 - Bắt đầu phát triển công nghệ đối tượng
 - Quy trình RUP, UML, các công cụ đầy đủ (ROSE, Jbuidner)
 - Kho dữ liệu, CSDL đối tượng, đa phương tiện
 - Định hướng sử dụng lại: thành phần, mẫu, framework

CÁC GIAI ĐOẠN PHÁT TRIỂN

- ~80s (nửa sau)
 - Phát triển các mô hình quản lý
 - Các chuẩn được công nhận (CMM, ISO)
 - Nhiều mô hình tổ chức làm phần mềm được đề xuất
 - Nhiều công cụ trợ giúp quản lý dự án được hoàn thiện

MÔ HÌNH PHÁT TRIỂN PHẦN MỀM NÓI CHUNG

MÔ HÌNH PHÁT TRIỂN PHẦN MỀM NÓI CHUNG

- Là các hoạt động từ khi được đặt hàng, phát triển, sử dụng đến khi loại bỏ nó
- Các giai đoạn chính
 - Giao tiếp: thu thập, phân tích và đặc tả yêu cầu
 - Lập kế hoạch: lập kế hoạch về phát triển PM, ước lượng rủi ro
 - Mô hình hóa: thiết kế hệ thống/mức cao/mức thấp
 - Xây dựng: lập trình + kiểm thử
 - Triển khai = chuyển giao + bảo trì

MÔ HÌNH PHÁT TRIỂN PHẦN MỀM NÓI CHUNG

- Có nhiều mô hình phát triển PM
 - Mô hình thác nước
 - Mô hình tiến hóa
 - Mô hình xoắn ốc
 - Mô hình phát triển theo nguyên tắc Agile, v.v.
- Các mô hình phát triển PM tuy có quy trình khác nhau, nhưng nếu xét về mặt từng bước chính nhỏ, thì đều có các giai đoạn đã nêu

NHỮNG KHÓ KHĂN TRONG PHÁT TRIỂN PM

Những khó khăn trong phát triển PM

- (1) Không có phương pháp mô tả rõ ràng định nghĩa yêu cầu của người dùng (khách hàng), sau khi bàn giao sản phẩm dễ phát sinh những trục trặc
- (2) Với những phần mềm quy mô lớn, tài liệu đặc tả đã cố định thời gian dài, do vậy khó đáp ứng nhu cầu thay đổi của người dùng một cách kịp thời trong thời gian đó
- (3) Nếu không có Phương pháp luận thiết kế nhất quán mà thiết kế theo cách riêng (của công ty, nhóm), thì sẽ dẫn đến suy giảm chất lượng phần mềm (do phụ thuộc quá nhiều vào con người)

Những khó khăn trong phát triển PM (2)

- (4) Nếu không có chuẩn về làm tài liệu trong quy trình sản xuất phần mềm, thì những đặc tả không rõ ràng sẽ làm giảm chất lượng phần mềm
- (5) Nếu không có giải pháp đảm bảo chất lượng của từng giai đoạn mà chỉ kiểm thử ở giai đoạn cuối và phát hiện ra lỗi, thì thường bàn giao sản phẩm không đúng hạn
- (6) Nếu coi trọng việc lập trình hơn khâu thiết kế thì thường dẫn đến làm giảm chất lượng phần mềm
- (7) Nếu coi thường việc tái sử dụng phần mềm (software reuse), thì năng suất lao động sẽ giảm

Những khó khăn trong phát triển PM (3)

- (8) Phần lớn trong quy trình phát triển phần mềm có nhiều thao tác do con người thực hiện, do vậy năng suất lao động thường bị giảm
- (9) Không chứng minh được tính đúng đắn của phần mềm, do vậy độ tin cậy của phần mềm sẽ giảm
- (10) Chuẩn về một phần mềm tốt không thể đo được một cách định lượng, do vậy không thể đánh giá được một hệ thống đúng đắn hay không
- (11) Công việc bảo trì kéo dài là hệ lụy lớn với tổ chức

Những khó khăn trong phát triển PM (4)

- (12) Quản lý dự án lỏng lẻo kéo theo quản lý lịch trình cũng không rõ ràng
- (13) Không có tiêu chuẩn để ước lượng nhân lực và dự toán sẽ làm kéo dài thời hạn và vượt kinh phí của dự án

Đây là những vấn đề phản ánh các khía cạnh khủng hoảng phần mềm, hãy tìm cách nỗ lực vượt qua để tạo ra phần mềm tốt!

Thực trạng các dự án CNTT ở VN

- Nhỏ lẻ, manh mún, thiếu thiết kế tổng thể
 - Các phần mềm trong một cơ quan không giao tiếp được với nhau
- Chất lượng thấp
 - Không đủ chức năng, khó sử dụng
 - Nhiều lỗi
 - ...
- Khó/không thể nâng cấp/bảo trì

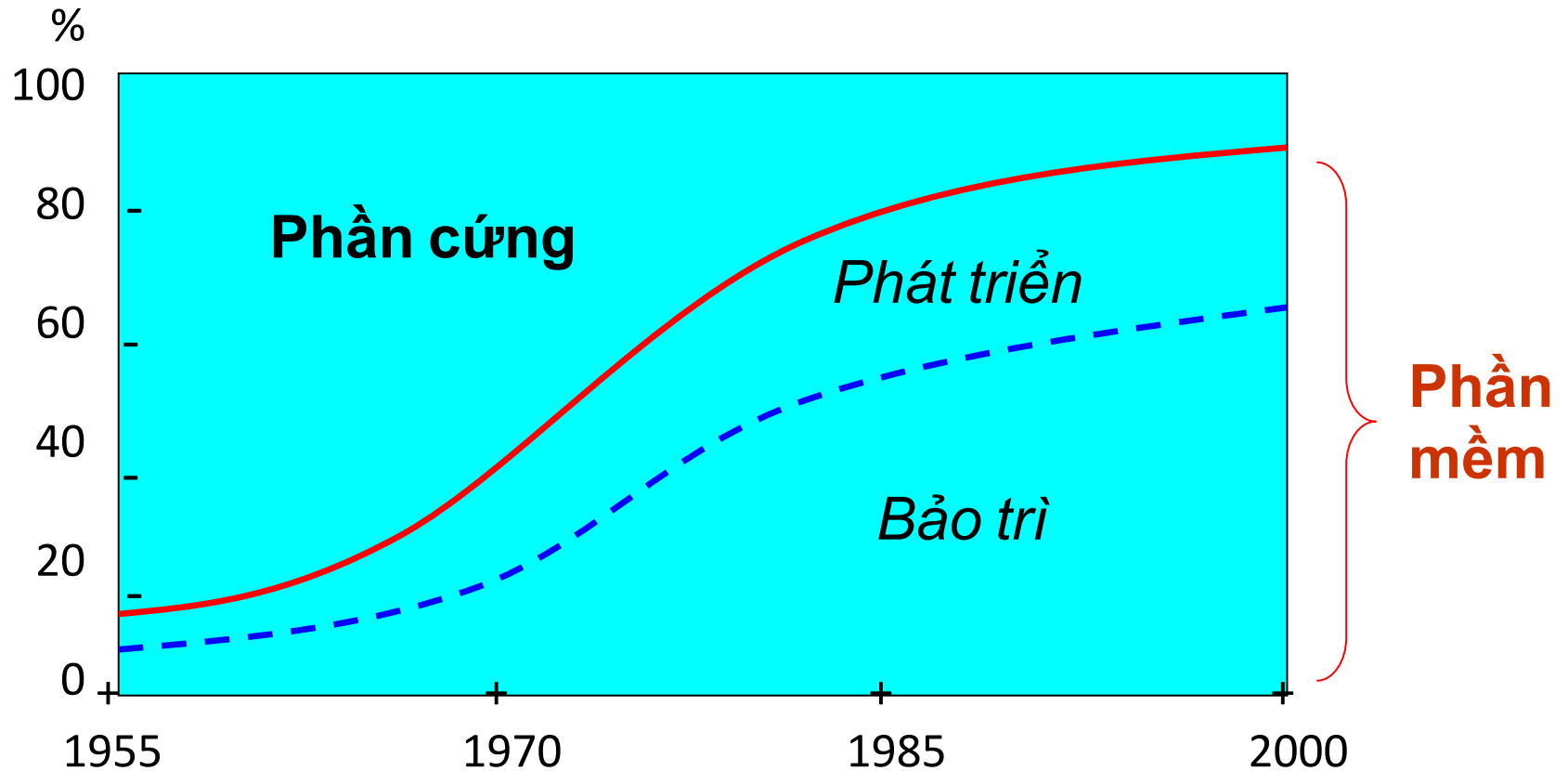
Thống kê của Standish Group (2006)

- Có tới 50% trong số các dự án phần mềm thất bại
- Chỉ có 16.2% dự án là hoàn thành đúng hạn và nằm trong giới hạn ngân sách, đáp ứng tất cả tính năng và đặc tính như cam kết ban đầu
- Có 52.7% dự án được hoàn thành và đi vào hoạt động nhưng không hoàn thành đúng hạn và bội chi, thêm nữa không đáp ứng đầy đủ tính năng và đặc tính như thiết kế ban đầu
- Và có 31.1% dự án thất bại trước khi được hoàn thành

-> hơn 83.8% dự án thất bại hoặc không đáp ứng những yêu cầu ban đầu

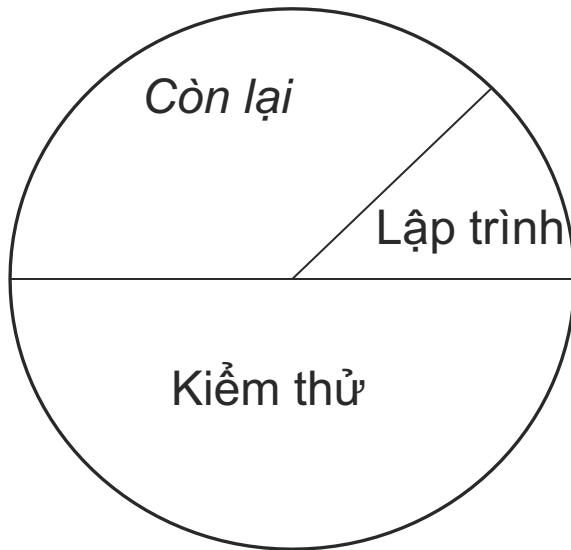
CHI PHÍ PHÁT TRIỂN

Chi phí cho Phần cứng và Phần mềm



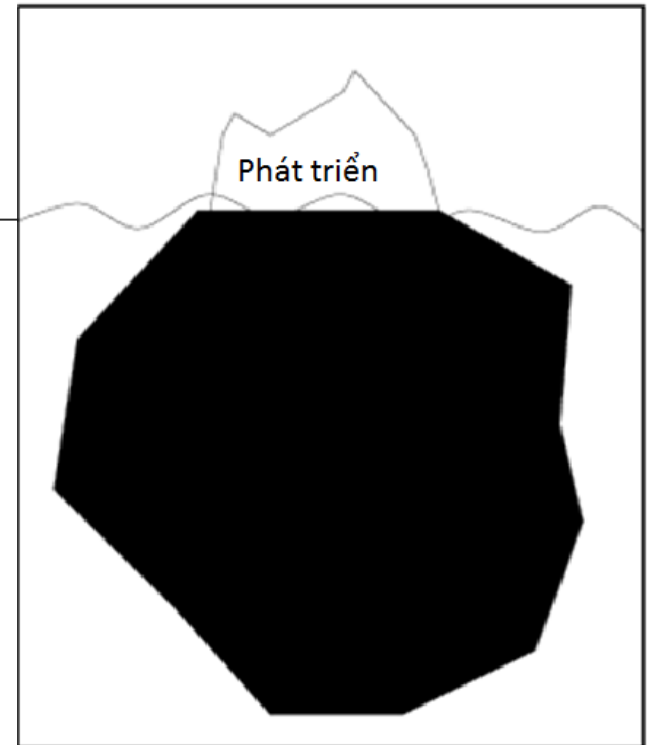
Chi phí cho các pha

Chi phí phát triển



Còn lại

Bảo trì



- Chi phí phát triển (giao tiếp + lập kế hoạch + mô hình hóa + lập trình + chuyển giao) chỉ là phần nổi của tảng băng chìm!
- Tồn kém ở KIỂM THỬ + BẢO TRÌ

SOFTWARE MYTH

Management myths

Managers with software responsibility, like managers in most disciplines, are often under pressure to maintain budgets, keep schedules from slipping, and improve quality

Myth: *If we get behind schedule, we can add more programmers and catch up (sometimes called the “Mongolian horde” concept).*

Reality: Software development is not a mechanistic process like manufacturing. In the words of Brooks [Bro95]: “adding people to a late software project makes it later.” At first, this statement may seem counterintuitive. However, as new people are added, people who were working must spend time educating the newcomers, thereby reducing the amount of time spent on productive development effort. People can be added but only in a planned and well-coordinated manner.

Customer myths

- In many cases, the customer believes myths about software because software managers and practitioners do little to correct misinformation
- Myths lead to false expectations (by the customer) and, ultimately, dissatisfaction with the developer

Myth: *Software requirements continually change, but change can be easily accommodated because software is flexible.*

Reality: It is true that software requirements change, but the impact of change varies with the time at which it is introduced. When requirements changes are requested early (before design or code has been started), the cost impact is relatively small.¹⁶ However, as time passes, the cost impact grows rapidly—resources have been committed, a design framework has been established, and change can cause upheaval that requires additional resources and major design modification.

Practitioner's myths

- In many cases, the customer believes myths about software because software managers and practitioners do little to correct misinformation
- Myths lead to false expectations (by the customer) and, ultimately, dissatisfaction with the developer

Myth: *Once we write the program and get it to work, our job is done.*

Reality: Someone once said that “the sooner you begin ‘writing code,’ the longer it’ll take you to get done.” Industry data indicate that between 60 and 80 percent of all effort expended on software will be expended after it is delivered to the customer for the first time.

Đạo đức nghề nghiệp

- Tính chuyên nghiệp
- Quy tắc đạo đức và ứng xử nghề nghiệp
- Các vấn đề về luật pháp

HỎI & ĐÁP

